

Documento Técnico

Etapas 1 — Construcción de la Base de Conocimiento

CODEFEST AD ASTRA 2026

Equipo Laka

Juan José Vélez Arcila

Julián Ojeda Avilés

Emanuel Vahos Villa

Manuela Galeano Chica

Institución Universitaria ITM

Medellín, Colombia

13 de agosto de 2026

1. Presentación

Somos el **Equipo Laka**, conformado por Juan José Vélez Arcila, Julián Ojeda Avilés, Emanuel Vahos Villa y Manuela Galeano Chica, estudiantes de la **Institución Universitaria ITM**, en Medellín, Colombia. Participamos en el CODEFEST AD ASTRA 2026 con este proyecto, que construye la base de conocimiento vectorial de la Etapa 1 y el módulo de recuperación que la consulta, distribuyendo el trabajo entre las etapas de extracción y limpieza del corpus, chunking, codificación e indexación, y el módulo de recuperación que se describe en este documento.

2. Descripción del problema

El CODEFEST AD ASTRA 2026 propone el desarrollo de una arquitectura inteligente multiagente orientada al análisis estratégico de fenómenos con impacto en el dominio aéreo y espacial, a partir de fuentes abiertas provenientes de observatorios y centros de pensamiento colombianos, regionales y globales. El reto se organiza en torno a tres fenómenos: (F1) inteligencia artificial e innovación en entornos militares, (F2) seguridad espacial y congestión de la órbita baja terrestre, y (F3) dinámicas territoriales en América Latina con implicaciones de seguridad y gobernanza. La Etapa 1, que es la fase virtual del reto y la que atañe a este documento, exige construir una **base de conocimiento vectorial** a partir del corpus documental provisto (PDF, HTML, JSON, CSV/XLSX, imágenes y mapas .pbf) y evaluarla exclusivamente por la **calidad de recuperación**: qué tan relevantes son los documentos y fragmentos que el sistema retorna ante 50 consultas en lenguaje natural. Esta base servirá como infraestructura para un asistente conversacional en la Etapa 2 (presencial), pero en esta etapa el criterio de evaluación es únicamente la recuperación de información, sin intervención de modelos generativos en ningún punto del proceso.

3. Arquitectura general

El pipeline sigue el flujo de extremo a extremo de la Figura 1 del spec: extracción y limpieza por fenómeno → chunking semántico → codificación con encoder → indexación en FAISS → recuperación. Las tres primeras etapas producen los artefactos persistidos en el repositorio (`data/clean/*.jsonl`, `entrega/base_vectorial/`); la última es `entrega/generador.py`, el script reproducible que consume esos artefactos y no construye nada nuevo.

4. Extracción y limpieza

Cada fenómeno tiene su propio script de extracción (`src/f{1,2,3}_limpieza_extraccion.py`), autocontenidos a propósito para no acoplar su evolución. Los tres comparten limpieza de caracteres de control, reparación de guionado a fin de línea, eliminación de cabeceras/pies repetidos y detección de idioma (`langdetect`).

Divergen donde el corpus lo exige: F1 y F3 extraen PDF con PyMuPDF en modo *texto plano* (una línea por línea visual, colapsando todo el espaciado); F2 usa el modo *blocks*, que preserva los saltos de párrafo como separadores y añade OCR sobre imágenes sueltas (incluyendo un archivo AVIF, con un reencodeo a PNG en memoria porque `pytesseract` no decodifica AVIF directamente) y una reducción de catálogos de scraping a `título | año | page_url`. F3 añade lectura robusta de CSV con detección de separador/codificación y extracción de teselas vectoriales Mapbox (.pbf), leyendo capas y propiedades como pares atributo:valor.

Un conjunto de documentos en idiomas fuera de español/inglés/portugués (traducciones oficiales del mismo reporte publicadas por UNOOSA, SWF y SIPRI) se excluye por una lista explícita de `doc_id` verificados a mano, no por `idioma_detectado`: `langdetect` produce falsos positivos frecuentes en texto

corto o técnico (español marcado como catalán, alemán o rumano en fuentes de CENIA y en datos .pbf) y en contenido no lingüístico. Esos falsos positivos se retienen explícitamente en vez de excluirse — solo se excluyen los casos verificados de idioma genuinamente distinto.

5. Estrategia de chunking y justificación

(Sección 3 del spec)

5.1 Estrategia elegida: semántica con superposición

Se implementó la estrategia *semántica con superposición* (Sección 3.2): en vez de trocear por tamaño fijo, se detectan cambios de tema calculando la distancia coseno entre embeddings de oraciones consecutivas (cada oración se embebe junto con una ventana de contexto de ± 1 oración, para estabilizar el punto de corte) y se corta donde esa distancia supera el percentil 95 de las distancias del propio documento — un umbral adaptativo por documento, no global. Los grupos semánticos resultantes por debajo de un mínimo de palabras se fusionan con el siguiente para evitar fragmentación excesiva.

5.2 Restricciones duras

Sobre el agrupamiento semántico se imponen dos restricciones que nunca se relajan:

- **Complejidad lingüística** (Sección 3.3 del spec): los cortes solo ocurren en fronteras de oración (tokenización con NLTK, adaptada por idioma detectado). Para formatos tabulares (csv/xlsx/pbf) se usa el salto de línea como frontera en vez de NLTK: Punkt trata un punto tras un número como parte del número, así que un CSV completo se tokenizaba como una sola oración gigante.
- **Límite de 250 palabras** (Sección 9.2.1 del spec): un grupo semántico que excede el límite se subdivide empaquetando oraciones, igual que un chunking por tamaño fijo, pero solo como mecanismo de contención — nunca como estrategia primaria.

5.3 Caso patológico: oraciones que exceden el límite por sí solas

Documentos de política pública con cláusulas largas, o texto tabular mal segmentado por el tokenizador, producen “oraciones” que superan las 250 palabras sin ningún punto de corte oracional disponible. Una heurística de densidad de conectores gramaticales (es/pt/en) distingue prosa genuina de artefactos no lingüísticos: la prosa se corta en el separador de cláusula (coma, punto y coma o dos puntos) más cercano al límite; los artefactos se cortan por bloques de palabras. Cada partición forzada se registra en un archivo de auditoría para trazabilidad.

Como red de seguridad final, cada chunk se re-verifica contra el *tokenizer real* del encoder (no un proxy por palabras): el español técnico infla hasta $2\times$ en subword tokens, así que un chunk de ≤ 250 palabras puede en teoría exceder la ventana de 512 tokens del encoder. Si ocurre, se vuelve a partir por conteo de tokens exacto antes de escribirse en la metadata.

5.4 Resultado medido

Métrica	Valor
Chunks totales	330,416
Documentos únicos (doc_id)	1,780
Palabras por chunk — mínimo / mediana / p90 / máximo	1 / 140 / 228 / 250
Chunks que exceden 250 palabras	0

Ningún chunk excede el límite: la restricción dura de la Sección 9.2.1 se cumple por construcción, no por post-filtrado. La distribución por fenómeno es $F1 = 200,780$, $F2 = 51,859$, $F3 = 77,777$ chunks, sobre

1,780 documentos únicos (454 en F1, 449 en F2, 877 en F3); la distribución por formato muestra que más de la mitad del corpus (53 %) proviene de formatos tabulares (csv, xlsx, pbf).

6. Encoder y criterios de elección

(Sección 4 del spec)

6.1 Encoder seleccionado

`intfloat/multilingual-e5-small` (familia BERT, 117.65 M de parámetros, licencia MIT, dimensión 384, ventana de 512 tokens). Requiere prefijar el texto con `"query: "` (consultas) o `"passage: "` (documentos) antes de codificar — aplicado consistentemente en indexación y en recuperación, y verificado explícitamente (Sección 7).

6.2 Criterios (Sección 4.3)

- **Soporte multilingüe nativo:** el corpus mezcla español, inglés y portugués. Se verificó con una prueba de humo de tres consultas (una por fenómeno, cada una en un idioma distinto) que recupera correctamente entre idiomas: 5/5 resultados del fenómeno esperado en los tres casos.
- **Dimensionalidad moderada** (384): equilibrio entre costo de almacenamiento/búsqueda y expresividad, adecuado para un índice `IndexFlatIP` de 330 mil vectores servido en CPU.
- **Longitud máxima de entrada** (512 tokens): compatible con el límite de 250 palabras de los chunks, con margen para la inflación de subword tokenization verificada en el paso de chunking.
- **Licencia MIT:** sin restricciones para el uso en el reto.
- **Eficiencia computacional:** modelo “small” de la familia E5, dimensionado para correr en CPU dentro de los tiempos del reto (índice completo codificado sin GPU).

6.3 Comparación empírica

`src/benchmark_modelos.py` implementa una comparación empírica de cinco encoders candidatos contra el ground truth anotado del equipo, calculando `NDCG@10` y `F1@3` y combinándolos por Conteo de Borda (Sección 11.2):

- `paraphrase-multilingual-MiniLM-L12-v2`
- `multilingual-e5-small` (seleccionado)
- `paraphrase-multilingual-mpnet-base-v2`
- `BAAI/bge-m3`
- `multilingual-e5-base`

No se usaron múltiples encoders combinados (Sección 4.4): un único encoder multilingüe ya cubre los tres idiomas del corpus sin necesitar fusión de rankings (`CombSUM/CombMNZ/RRF`), lo que simplifica el pipeline de recuperación sin sacrificar cobertura de idioma.

7. Índice FAISS

(Sección 5 del spec)

`IndexFlatIP` sobre vectores L2-normalizados (equivalente a similitud coseno exacta, Sección 5.2), con 330,416 vectores de dimensión 384. Es la opción recomendada por el spec para el volumen de este corpus: búsqueda exacta sin pérdida de precisión, sin la complejidad de ajustar `nprobe` (IVF) o el uso de memoria de un grafo HNSW. El archivo resultante pesa ~508 MB; la codificación completa del corpus tomó 9.5 minutos sobre GPU (~580 chunks/segundo). El orden de inserción se mantuvo idéntico al orden de lectura de `metadata.jsonl`, garantizando que el id interno de FAISS coincida con el número de línea del

almacén de metadata (Sección 1.4).

Verificaciones realizadas sobre el índice entregado (`src/validar_indice.py`):

Chequeo	Resultado
Alineación metadata $\leftrightarrow$ índice	330,416 líneas == 330,416 vectores; la línea N corresponde al id interno N
Normalización	Muestra de 200 vectores, norma unitaria en todos
Identidad vector $\leftrightarrow$ texto	30 muestras: similitud coseno mínima = 1,000000 entre el vector guardado y el reembebido desde el texto
Humo multilingüe (es/en/pt)	5/5 resultados del fenómeno esperado en cada una de las tres consultas

El chequeo de identidad es la prueba más exigente: reconstruye el vector en una posición y lo compara contra el reembebido desde el texto de esa misma línea de metadata. Una similitud de 1.000000 descarta cualquier desalineación entre el orden de indexación y el orden del archivo de metadata.

8. Módulo de recuperación

(Sección 8 del spec)

`entrega/generador.py` implementa el flujo de la Sección 8.1 sin usar en ningún punto un modelo generativo (Sección 8.3): la consulta se codifica con el mismo encoder e idéntico prefijo E5, se normaliza, y se busca en FAISS. Toda decisión opera sobre vectores, puntuaciones coseno y metadata.

8.1 Lectura de consultas

El script acepta el archivo de consultas en varios formatos (`.jsonl/.json/.csv/.xlsx`), autodetectado por extensión y con alias flexibles de nombre de columna (`query_id/id/qid`, `query/pregunta/ texto`). El archivo `entrega/consultas.jsonl` se generó a partir del PDF oficial de los organizadores (`Extracto_Preguntas_50_v2.pdf`, en `data/raw/`) con un parser por expresión regular sobre el patrón `qNWN`, verificado línea por línea contra la lista de referencia usada durante el desarrollo: 50/50 idénticas. Se generó como paso explícito de preparación — necesario para que `generador.py` sea ejecutable de punta a punta, ya que el spec exige en la Sección 1.4 que el script “lea el archivo de consultas” sin fijar su formato, y sin él la Sección 1.4 también advierte que una entrega no reproducible se excluye de la evaluación.

8.2 Búsqueda y agregación a documento (Sección 8.6)

El tamaño del pool de candidatos k es dinámico: arranca en 30 y se duplica hasta 200 si hace falta, garantizando que toda consulta obtenga exactamente 3 documentos y 10 fragmentos — requisito estricto de la Sección 9.3.2, donde un conteo incorrecto se penaliza o descarta. Los candidatos se agrupan por `doc_id` y se agregan con **max pooling** (el score del fragmento más relevante representa al documento), una de las tres estrategias que nombra textualmente la Sección 8.6.

Esta elección no fue arbitraria: durante la validación se detectó un sesgo sistemático real, documentado en la Sección 9.1, y se probaron y descartaron dos alternativas antes de confirmar max pooling puro como la configuración final.

8.3 Límite de 250 palabras por fragmento (Sección 9.2.1)

Los chunks recuperados que superan 250 palabras se dividen en sub-fragmentos mediante tokenización de oraciones (NLTK) empaquetadas de forma voraz hasta el límite, preservando siempre fronteras oracionales completas; todas las sub-piezas de un mismo chunk comparten el mismo `chunk_id` y ocupan posiciones de ranking independientes, como exige el spec. En la práctica esta ruta **no se activa** sobre

el corpus actual (ningún chunk supera 250 palabras, Sección 3), pero se implementó y validó con texto sintético para no dejar un caso límite sin cubrir.

8.4 Autovalidación de salida

`generador.py` valida su propia salida al terminar: cuenta líneas de `resultados.jsonl`, exige exactamente 3 documentos y 10 fragmentos por consulta, y verifica que ningún fragmento exceda el límite de palabras — una red de seguridad barata contra errores de esquema antes de la entrega.

8.5 Observaciones sobre la salida y correcciones aplicadas

La entrega original de la Etapa 1 (13 de agosto de 2026) presentaba dos comportamientos medidos directamente sobre `resultados.jsonl`. Tras la clasificación a la Etapa 2, con margen para revisar la base antes del asistente conversacional, ambos se corrigieron en `entrega/generador.py` y se verificaron de nuevo sobre las 50 consultas reales contra el mismo índice FAISS:

- **Normalización de texto (corregido):** el 89 % de los fragmentos (445/500) arrastraba saltos de línea de la extracción de PDF a mitad de frase, mientras que el 100 % de los excertos anotados en el ground truth de referencia son de una sola línea. Como la relevancia se juzga por el *contenido* del campo `text` (Sección 10.2.1), se implementó `normalizar_texto()`, que colapsa saltos de línea y espacios repetidos antes de escribir cada fragmento — sin infringir el esquema (la Sección 9.2.1 ya autoriza modificar el texto de salida) y sin tocar el campo `texto` de `metadata.jsonl`, que la Tabla 1 exige sin modificaciones. Verificado sobre las 50 consultas reales: **445/500 (89.0 %) → 0/500 (0.0 %)**.
- **Deduplicación de fragmentos (corregido):** al no filtrar por similitud, 19 de las 50 consultas (38 %) tenían al menos dos fragmentos casi idénticos dentro de su propio top-10 — esperable dado que el corpus incluye documentos con filas de contenido muy repetitivo. Se implementó `es_casi_duplicado()` (similitud de secuencia de caracteres sobre el texto ya normalizado, umbral 0.92), integrada en `construir_fragmentos()`: un candidato casi idéntico a uno ya aceptado se descarta y el bucle de expansión de k (Sección 8.6) sigue recuperando candidatos hasta completar el top-10 con contenido distinto. Autorizado por la Sección 8.7 (post-filtro por contenido de metadata). Verificado sobre las 50 consultas reales: **19/50 (38.0 %) → 0/50 (0.0 %)**.

Ambas correcciones se validaron por reproducibilidad exacta: regenerar `resultados.jsonl` desde cero con `generador.py` produce las 50 consultas idénticas byte a byte al archivo entregado.

8.6 Contrato de invocación

Conforme a la Sección 1.5, `generador.py` se ejecuta con `python generador.py` sin argumentos desde la raíz de `entrega/`: `--indice` y `--consultas` tienen valores por defecto que coinciden con la estructura entregada (`base_vectorial/encoder_multilingual-e5-small` y `consultas.jsonl`, respectivamente, siguiendo el criterio de la Tabla 1 del spec), y producen `resultados.jsonl` de forma reproducible.

9. Evaluación y limitaciones

(Sección 10 del spec)

9.1 Experimentos de agregación: hipótesis probadas y descartadas

Durante la validación se detectó un sesgo sistemático: documentos con muchísimos chunks (p. ej. el Índice Latinoamericano de IA, hasta 975 chunks frente a una mediana de 7 en todo el corpus) dominaban el top-3 de consultas específicas incluso cuando existía un documento más pequeño y preciso con el contenido correcto — confirmado con la consulta “¿Qué desafíos plantea el uso de inteligencia artificial en las

operaciones espaciales?”, donde el sistema devolvió 0/3 documentos del fenómeno correcto pese a que existían 86 chunks de contenido de IA aplicada al dominio espacial en el corpus. La causa raíz: con max pooling, el score de un documento es el máximo entre sus chunks candidatos — el mejor resultado de N intentos—, y un documento con más chunks tiene estadísticamente más oportunidades de que uno de ellos puntúe alto por azar (estadística de valores extremos), sin que eso implique mayor relevancia real. Se probaron tres alternativas contra las 50 consultas reales (no contra casos aislados), usando como proxy la coincidencia de fenómeno entre documento y consulta, validado además con casos de depuración individuales antes de la corrida completa:

Hipótesis	Mecanismo	Resultado empírico (50 consultas)	Decisión
Penalización log	$\text{score} - \beta \log(n)$	F1: 85 % → 48 % (penaliza injustamente a documentos de tamaño típico de F1 frente a F3, mediana 2 vs. 14 chunks/doc)	Descartada
+ umbral (100)	$\text{score} - \beta \max(0, \log n - \log 100)$	— Sin cambio real (F1 igual en 48 %); total agregado 89.3 % → 78.7 % (−10.6 pp)	Descartada
Media de top- k	promedio de los 3 chunks más altos	Empeoró el caso diagnóstico: premió amplitud genérica sobre enfoque específico y certero	Descartada

Las tres empeoraron el resultado agregado frente a max pooling sin modificar, así que se revirtieron. La configuración final entregada usa max pooling puro.

9.2 Validación contra ground truth

El archivo FASE ORDENADA CODEFEST.xlsm, provisto por el equipo organizador, contiene relevancia verificada (pregunta, fragmento y documento fuente) para **7 de las 50 consultas** (q002, q003, q004, q033–q036; hojas F1 y F3, sin hoja para F2), cruzado contra el doc_id interno vía coincidencia normalizada del nombre de archivo (campo *fuentes*). Es una muestra pequeña que no reemplaza el NDCG@10/F1@3 oficial sobre las 50 consultas — cuyo ground truth no es público durante el reto — pero es evidencia real medida dos veces, con dos metodologías independientes:

- **Proxy de coincidencia** (top-3/top-10, sin ponderar por posición): 1/7 consultas (14 %) obtuvieron el documento correcto en el top-3; 3/7 (43 %) lo obtuvieron en el top-3 o entre los 10 fragmentos.
- **NDCG@10/F1@3 exactos** (Sección 10.2, recalculados con emparejamiento uno-a-uno entre fragmento y excerto para evitar que el DCG supere al IDCG): $\overline{\text{NDCG@10}} = 0,1429$, $\overline{\text{F1@3}} = 0,0714$ sobre las mismas 7 consultas.

Ambas metodologías apuntan en la misma dirección — desempeño débil sobre esta muestra pequeña, con margen real de mejora — lo cual da más confianza que cualquiera de las dos por separado. El patrón de falla de la Sección 9.1 se confirma en esta muestra independiente: las consultas que combinan el término genérico “inteligencia artificial” con un calificador técnico específico (“sistemas no tripulados”, “conflictos recientes”, “operaciones espaciales”) tienden a recuperar el Índice Latinoamericano de IA — un reporte extenso y genérico — en lugar de la fuente específica correcta (informes DAIO, CSIS), incluso cuando esta última existe en el corpus con contenido directamente relevante. Se identifica como **limitación conocida** del enfoque de recuperación puramente vectorial con un encoder compacto (117 M parámetros): el término genérico domina la señal semántica sobre el calificador específico.

Con una muestra de 7 consultas cualquier cifra puntual es sensible a 1–2 casos individuales; estos números deben leerse como evidencia direccional, no como una estimación precisa de NDCG@10/F1@3 sobre las 50 consultas oficiales.

10. Grafo de conocimiento

(Sección 7 del spec)

Como componente bonus se construyó un grafo de conocimiento (`base_vectorial/grrafo/grrafo.graphml`, formato GraphML, 452,379 nodos y 1,155,098 aristas) a partir de los mismos chunks indexados. Los nodos son entidades nombradas (tipos `PER`, `ORG`, `LOC`) con atributos de frecuencia y fenómeno(s) (1/2/3) en que aparecen; las aristas son relaciones de coocurrencia (`co_occurs_with`) ponderadas por número de coocurrencias, y conservan trazabilidad completa hacia `doc_ids` y `chunk_ids` de origen, más un ejemplo de evidencia textual cuando aplica. El grafo no participa en la recuperación en esta entrega (que sigue siendo puramente vectorial vía FAISS, sin LLMs) y se entrega como artefacto exploratorio complementario para análisis de co-ocurrencia de entidades entre fenómenos.

El archivo es XML bien formado y carga sin errores con `networkx.read_graphml` u otra herramienta compatible (Gephi, yEd, Neo4j): sin caracteres de control ilegales, sin ids de nodo duplicados y sin nodos vacíos.